

# Multi-Source Candidate Data Transformer - Design Doc

Goal: turn messy, multi-source candidate data into one clean, canonical, fully-traceable profile per candidate, where a wrong-but-confident value is treated as worse than an honest null.

## Pipeline

**detect -> extract -> normalize -> merge -> confidence -> project-to-output -> validate.** Detect picks a source-type-specific extractor by extension/URL pattern (never crashes on a missing or misnamed file). Extract produces loosely-typed *RawRecords* per source - extractors only fill in what they actually find, never invent values. Normalize applies deterministic formatting rules per field. Merge clusters *RawRecords* into one person and resolves conflicts (confidence is computed inline during merge, since confidence is a property of how a field was merged). Project reshapes the canonical record into whatever schema a runtime config asks for. Validate type-checks the final output before returning it.

## Canonical schema & normalization

Internal model keeps every (value, confidence, contributing sources) per field plus a flat provenance log - richer than any single output schema so projection never needs to re-touch raw sources. Dates -> YYYY-MM (year-only inputs default to 01, flagged as a warning). Countries -> ISO-3166 alpha-2 via a small alias map. Skills -> a canonical-name lookup table (e.g. "js"/"javascript" -> "JavaScript"). Phones -> a hand-rolled E.164 normalizer (no phone-number library was installable in this offline environment): strips formatting, assumes a configurable default country code (+91) when none is given, and never invents digits. This is a known limitation, called out below.

## Merge / conflict-resolution policy

**Match keys:** primary = normalized email (exact, case-insensitive); fallback (only when neither record has an email) = normalized full name - acknowledged as the weakest possible key. Records are clustered with union-find so transitive matches merge correctly.

**Source reliability weights** (used to break ties): recruiter\_csv 0.90, ats\_json 0.85, resume 0.80, linkedin 0.75, github 0.70, notes 0.50 (free text is least structured / most error-prone).

**Winner selection:** if all sources agree on a scalar field, value = that value, confidence = max source reliability + a small corroboration boost per extra agreeing source (capped at 1.0). If sources disagree, the highest-reliability source's value wins, but confidence is penalized (x0.7) to reflect the unresolved conflict - and every competing value is still written to *provenance*, never silently dropped, so a human can audit the decision. List fields (emails, phones, skills) are unioned + deduped rather than single-winner, since a candidate can legitimately have more than one of each.

## Runtime custom-output config (projection + validation)

The internal canonical record is first rendered into the fixed DEFAULT schema dict - the one clean boundary between internal modeling and the output contract. The projection layer only ever reads from this dict via a small path resolver supporting dotted paths, list indices (*emails[0]*), and a map operator (*skills[].name*) to pull a field out of every item in a list. Per requested field, the config can rename/remap it (*from*), force a normalization (E.164 / canonical), and the top-level config can toggle confidence/provenance and choose an *on\_missing* policy: null (default), omit, or error (raises after collecting every missing required field, so all problems surface at once). The projected output is then re-validated against the same config's declared types before being returned.

## Edge cases handled

- Same candidate across two sources with the same email but a conflicting job title -> CSV (higher reliability) wins, but the ATS value is preserved in provenance, not discarded.
- Malformed phone number (e.g. "not-a-phone") -> normalizer returns null + a logged warning rather than guessing or crashing.
- Completely empty CSV row / empty notes file / missing file path -> zero records produced for that source, run continues; never raises.
- Malformed JSON (ATS blob) -> caught, source skipped with a warning, rest of the pipeline proceeds.
- Free-text recruiter notes with no email, only a name -> still extracted and merged via the weaker name-based fallback key (flagged as the riskiest path in the design, not fully solved).

## Deliberately descope under time pressure

Full phone-number-library-grade validation; an NLP/LLM-based extractor for notes/resumes (used deterministic regex instead, trading recall for explainability and reproducibility); a real skill taxonomy service (used a small seed dictionary); LinkedIn extraction (no public API without auth, descope in favor of GitHub + resume + notes, which already satisfy the "one structured + one unstructured" minimum several times over).